

Backend Java Viva Questions

Accenture Full Stack + GenAI Viva preparation — Backend Java only. Covers Java fundamentals through JDBC, SQL, JPA and Hibernate. Excludes Spring, frontend, Docker, Agile, Kubernetes and other non-backend topics.

1. Java Fundamentals

1. What is Java?
2. What are the main features of Java?
3. Why is Java platform independent?
4. What is JVM?
5. What is JRE?
6. What is JDK?
7. Difference between JDK, JRE and JVM?
8. What is bytecode?
9. How does a Java program execute?
10. Why is Java called an object-oriented language?
11. Is Java 100% object-oriented?
12. What is a class?
13. What is an object?
14. What is a reference variable?
15. What are primitive data types?
16. What are non-primitive data types?
17. What is type casting?
18. What is widening and narrowing?
19. What are local, instance and static variables?
20. What is the default value of instance variables?

2. OOPs

21. What are the four pillars of OOP?
22. What is encapsulation?
23. How do you achieve encapsulation in Java?
24. What is inheritance?
25. What are the types of inheritance?
26. Does Java support multiple inheritance?
27. Why doesn't Java support multiple inheritance through classes?
28. What is polymorphism?
29. Compile-time vs runtime polymorphism?
30. What is method overloading?
31. What is method overriding?

- 32. Rules for method overriding?
- 33. Can a static method be overridden?
- 34. Can a private method be overridden?
- 35. Can a final method be overridden?
- 36. What is abstraction?
- 37. How do you achieve abstraction?
- 38. Abstract class vs interface?
- 39. Can an abstract class have a constructor?
- 40. Can an interface have methods with implementation?
- 41. What is default method in an interface?
- 42. What is a functional interface?
- 43. Can an interface extend another interface?
- 44. Can a class implement multiple interfaces?
- 45. What is the diamond problem?

3. Constructors / Keywords

- 46. What is a constructor?
- 47. Types of constructors?
- 48. Constructor vs method?
- 49. Can a constructor be overloaded?
- 50. Can a constructor be overridden?
- 51. What is this?
- 52. What is super?
- 53. this() vs super()?
- 54. What is static?
- 55. What is a static block?
- 56. What is final?
- 57. final vs finally vs finalize?
- 58. What is an access modifier?
- 59. public vs private vs protected vs default?
- 60. What is a package?

4. String

- 61. Why is String immutable?
- 62. What is String Pool?
- 63. String literal vs new String()?
- 64. String vs StringBuilder?
- 65. StringBuilder vs StringBuffer?
- 66. Why is StringBuffer thread-safe?

- 67. What is == vs equals()?
- 68. Why should we override equals()?
- 69. Why should hashCode() be overridden with equals()?
- 70. What happens when you concatenate Strings?

5. Exception Handling

- 71. What is an exception?
- 72. Exception vs Error?
- 73. Checked vs unchecked exception?
- 74. What is exception handling?
- 75. Explain try, catch, finally.
- 76. What is throw?
- 77. What is throws?
- 78. throw vs throws?
- 79. Can we have multiple catch blocks?
- 80. Can we have try without catch?
- 81. Can finally execute if an exception occurs?
- 82. What happens if finally itself throws an exception?
- 83. What is a custom exception?
- 84. How do you create a custom exception?
- 85. What is try-with-resources?

6. Collections

- 86. What is the Collection Framework?
- 87. Collection vs Collections?
- 88. List vs Set vs Map?
- 89. ArrayList vs LinkedList?
- 90. ArrayList vs Vector?
- 91. HashSet vs LinkedHashSet?
- 92. HashSet vs TreeSet?
- 93. HashMap vs Hashtable?
- 94. HashMap vs LinkedHashMap?
- 95. HashMap vs TreeMap?
- 96. How does HashMap work internally?
- 97. What is hashing?
- 98. What is a collision?
- 99. How does HashMap handle collisions?
- 100. Can HashMap have null keys?
- 101. Why does HashMap allow only one null key?

- 102. What is ConcurrentHashMap?
- 103. Iterator vs ListIterator?
- 104. What is fail-fast?
- 105. What is fail-safe?
- 106. Comparable vs Comparator?
- 107. What are generics?
- 108. Why do we use generics?

7. Java 8

- 109. What are the major features of Java 8?
- 110. What is a lambda expression?
- 111. Why were lambda expressions introduced?
- 112. What is a functional interface?
- 113. What is @FunctionalInterface?
- 114. Predicate vs Consumer vs Supplier vs Function?
- 115. What is Stream API?
- 116. Stream vs Collection?
- 117. map() vs filter()?
- 118. map() vs flatMap()?
- 119. What is reduce()?
- 120. What is collect()?
- 121. What is forEach()?
- 122. What is method reference?
- 123. What is Optional?
- 124. Why do we use Optional?
- 125. What are default methods?
- 126. What are static methods in interfaces?

8. Multithreading

- 127. What is a thread?
- 128. Process vs thread?
- 129. How can you create a thread?
- 130. Thread vs Runnable?
- 131. Runnable vs Callable?
- 132. What is the thread lifecycle?
- 133. What is synchronization?
- 134. Why do we need synchronization?
- 135. What is a race condition?
- 136. What is deadlock?

- 137. How can deadlock be prevented?
- 138. What is ExecutorService?
- 139. What is Future?
- 140. What is a daemon thread?

9. JDBC + SQL

- 141. What is JDBC?
- 142. Why do we use JDBC?
- 143. Explain the JDBC architecture.
- 144. What are the steps to connect Java to a database?
- 145. What is JDBC Driver?
- 146. Statement vs PreparedStatement?
- 147. Why is PreparedStatement preferred?
- 148. What is CallableStatement?
- 149. What is ResultSet?
- 150. What is SQL injection?
- 151. How can SQL injection be prevented?
- 152. What is a transaction?
- 153. What are commit and rollback?
- 154. What is connection pooling?
- 155. What is ACID?
- 156. What is a primary key?
- 157. What is a foreign key?
- 158. What is normalization?
- 159. What is an index?
- 160. INNER JOIN vs LEFT JOIN?

10. JPA / Hibernate

- 161. What is ORM?
- 162. What is JPA?
- 163. What is Hibernate?
- 164. JPA vs Hibernate?
- 165. What is an Entity?
- 166. What is @Entity?
- 167. What is @Id?
- 168. What is @GeneratedValue?
- 169. What is @Table?
- 170. What is @Column?
- 171. What is EntityManager?

- 172.** What is persistence context?
- 173.** What is JPQL?
- 174.** JPQL vs SQL?
- 175.** What is lazy loading?
- 176.** What is eager loading?
- 177.** Lazy vs eager loading?
- 178.** What is cascading?
- 179.** What is @OneToOne?
- 180.** What is @OneToMany?
- 181.** What is @ManyToOne?
- 182.** What is @ManyToMany?
- 183.** What is @JoinColumn?
- 184.** What is dirty checking?
- 185.** What is first-level cache?